Київський Національний Університет імені Тараса Шевченка
Факультет Комп’ютерних Наук та Кібернетики

Лабораторна Робота №1 (a, d)

З Курсу:

“ Основи об'єктно-орієнтованого програмування”

По темі «Моделювання з використанням UML»

Виконав студенти
Групи ІПС-22

Бойчук Ростислав

2026

Загальний опис проєкту

У межах виконання лабораторної роботи було реалізовано програмну систему для роботи з матрицями та виконання базових операцій лінійної алгебри.

Проєкт складається з двох версій:

1. **Стара версія** — реалізована на мові програмування C у процедурному стилі. Архітектура базується на глобальних змінних та функціях, що працюють із масивами фіксованого розміру.
2. **Нова версія** — реалізована на мові C++ з використанням об'єктно-орієнтованого підходу, інкапсуляції, поліморфізму та патерну проектування Strategy.

Метою рефакторингу було покращення архітектури системи, забезпечення тестованості коду, розширюваності та відповідності принципам SOLID.

Функціональні можливості системи

Система підтримує наступні операції:

- Додавання матриць
- Віднімання матриць
- Множення матриць (класичний алгоритм)
- Множення матриць (алгоритм Штрассена)
- Множення матриці на скаляр
- Транспонування матриці
- Інтерактивне меню користувача
- Автоматизоване unit-тестування

Користувач вводить розміри матриць та їх елементи через консольний інтерфейс, після чого система виконує обрану операцію.

Нефункціональні вимоги

Система повинна відповідати наступним вимогам:

- Модульна архітектура
- Інкапсуляція внутрішньої реалізації
- Використання інтерфейсів
- Підтримка розширення алгоритмів множення
- Відокремлення логіки від користувацького інтерфейсу
- Підтримка автоматизованого тестування
- Простота підтримки та масштабування
- Відсутність дублювання логіки (принцип DRY)

Архітектура системи

Нова версія системи побудована за об’єктно-орієнтованою архітектурою.

Основні компоненти:

Matrix

Клас, що відповідає за:

- Зберігання даних матриці
- Доступ до елементів
- Транспонування

Використовується контейнер std::vector для зберігання даних.

MatrixOperations

Клас, що реалізує статичні методи:

- add()
- subtract()
- scalarMultiply()

Відповідає за базові математичні операції.

IMatrixMultiplier

Інтерфейс для алгоритмів множення матриць.

Містить абстрактний метод:

multiply(A, B)

ClassicalMultiplier

Реалізація класичного алгоритму множення матриць.

StrassenMultiplier

Реалізація алгоритму Штрассена.

Алгоритм використовується при великих розмірах матриць.

Для малих матриць відбувається автоматичний перехід до класичного алгоритму.

Menu

Відповідає за користувацький інтерфейс.

- Виводить меню
- Отримує вибір користувача
- Перевіряє коректність введення

Controller

Керує логікою роботи системи.

- Приймає вибір користувача
- Викликає відповідні операції
- Обирає алгоритм множення через Strategy

UML-моделювання

У межах роботи було побудовано наступні UML діаграми:

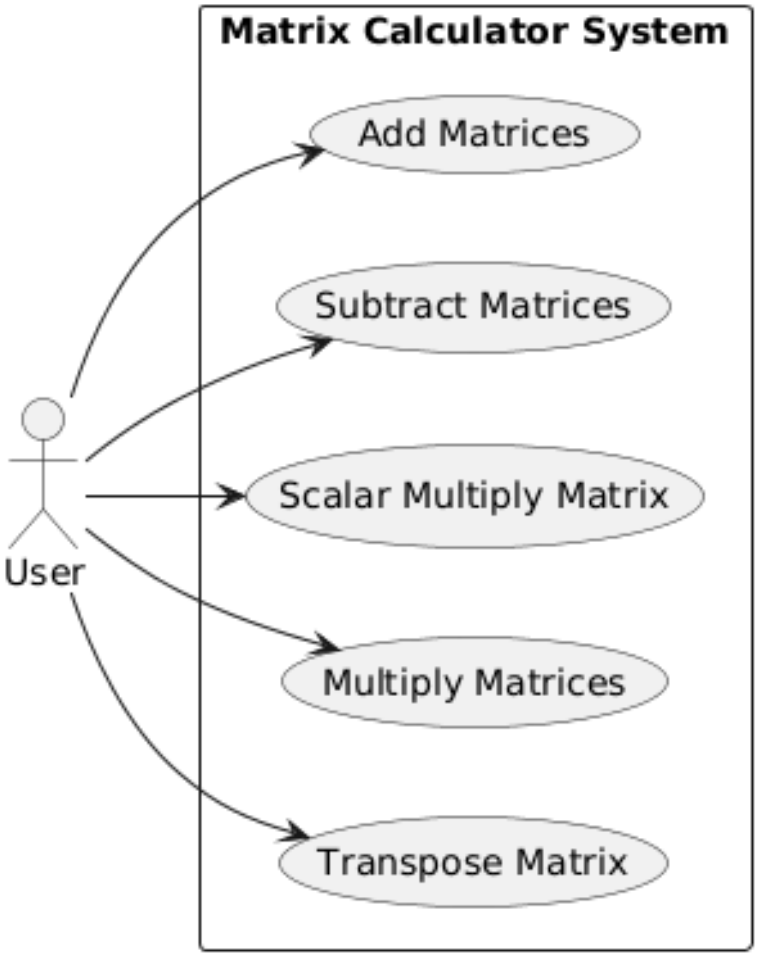
Use Case Diagram

Описує взаємодію користувача з системою.

Актор: User

Варіанти використання:

- Виконати додавання
- Виконати віднімання
- Виконати множення
- Виконати транспонування
- Обрати алгоритм множення



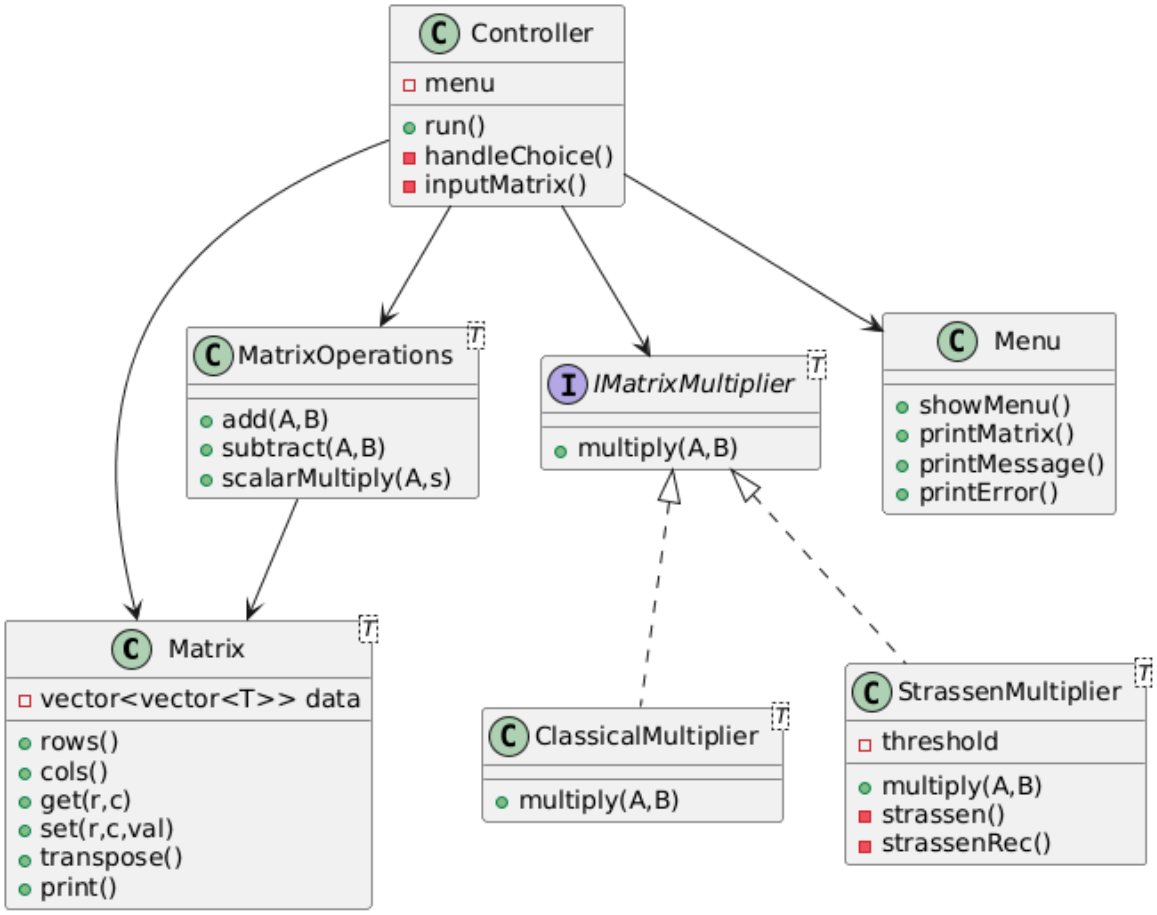
Class Diagram

Відображає структуру класів:

- Matrix
- MatrixOperations
- IMatrixMultiplier
- ClassicalMultiplier
- StrassenMultiplier
- Controller
- Menu

Показано:

- Асоціації
- Наслідування
- Реалізацію інтерфейсу

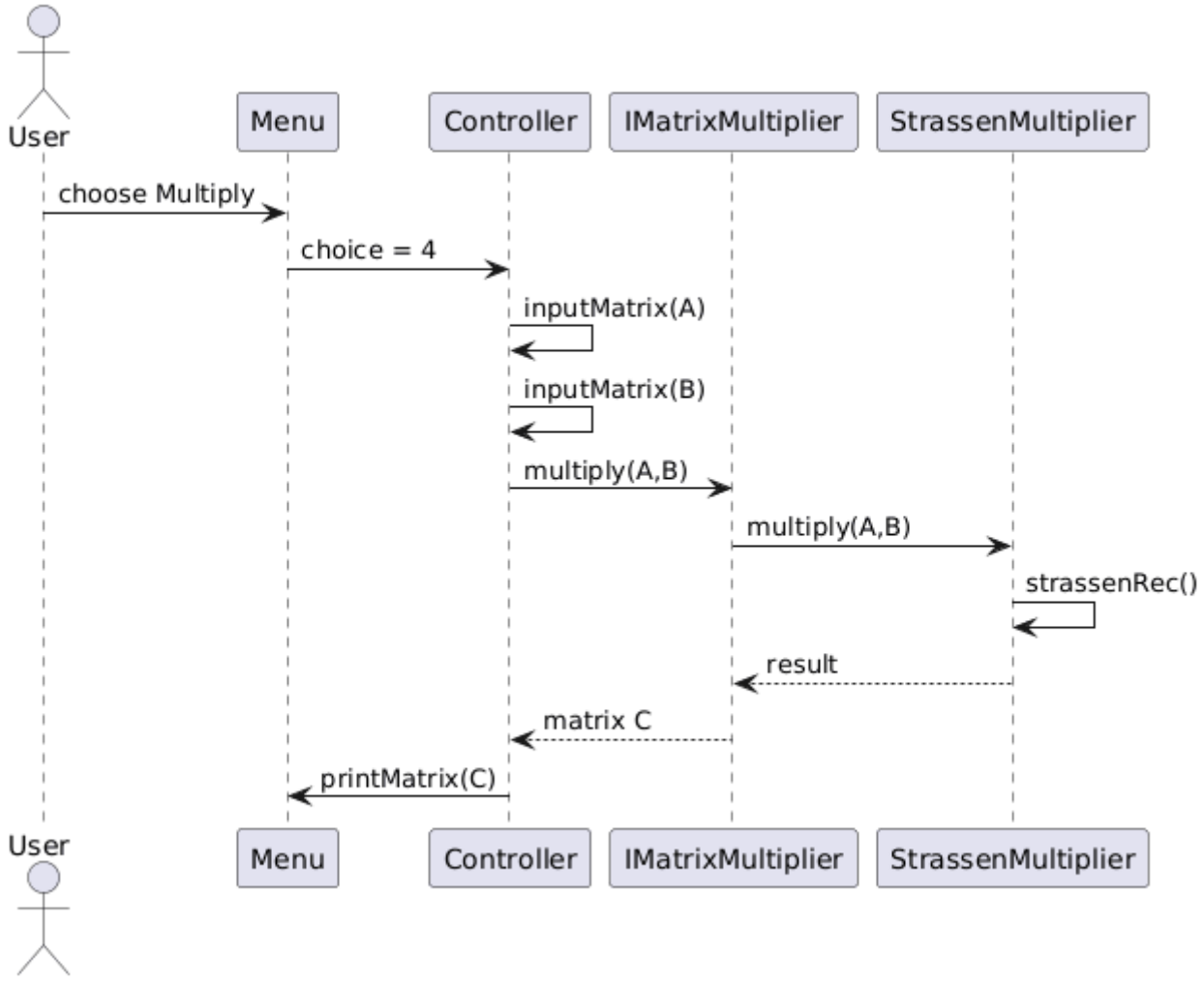


Sequence Diagram

Описує послідовність викликів при виконанні операції множення:

User → Menu → Controller → Multiplier → Matrix → Result

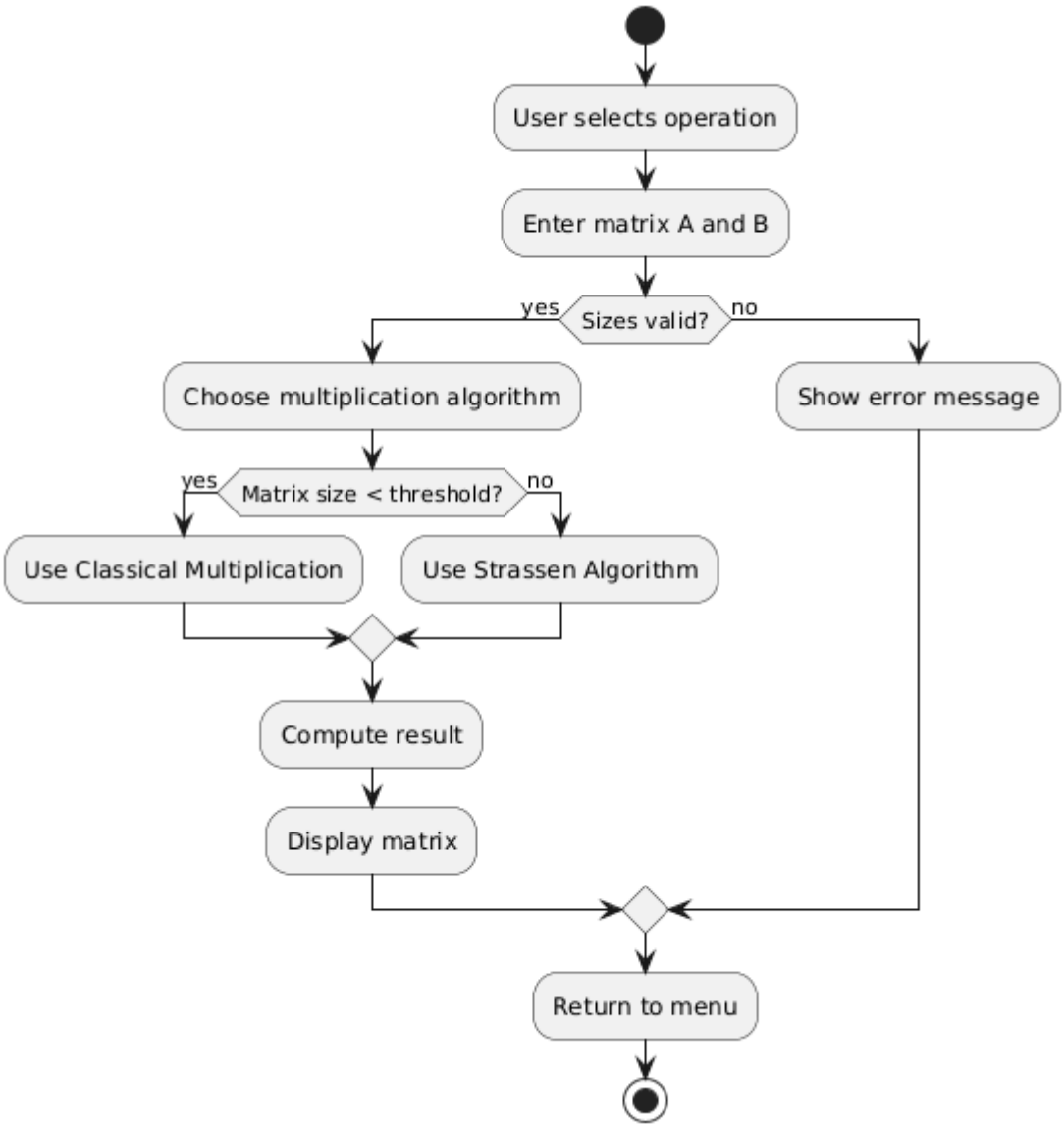
Показує взаємодію між об’єктами під час виконання алгоритму.



Activity Diagram

Описує алгоритм виконання операції:

1. Користувач обирає операцію
2. Вводить матриці
3. Перевіряється коректність розмірів
4. Обирається алгоритм
5. Обчислюється результат
6. Виводиться результат

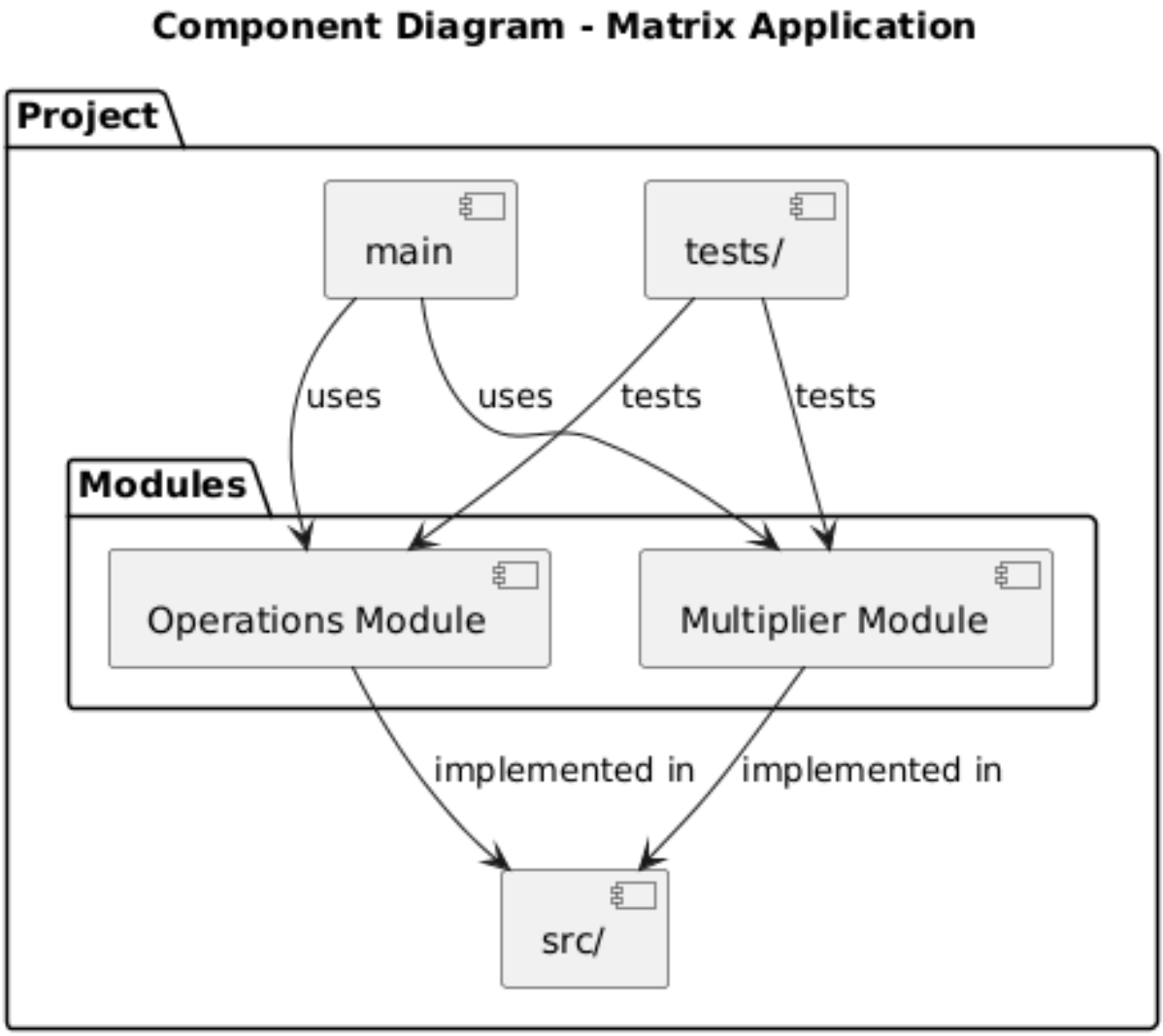


Component Diagram

Відображає структуру програмних модулів:

- src/
- tests/
- main
- Multiplier module
- Operations module

Показує взаємозв’язки між частинами системи.

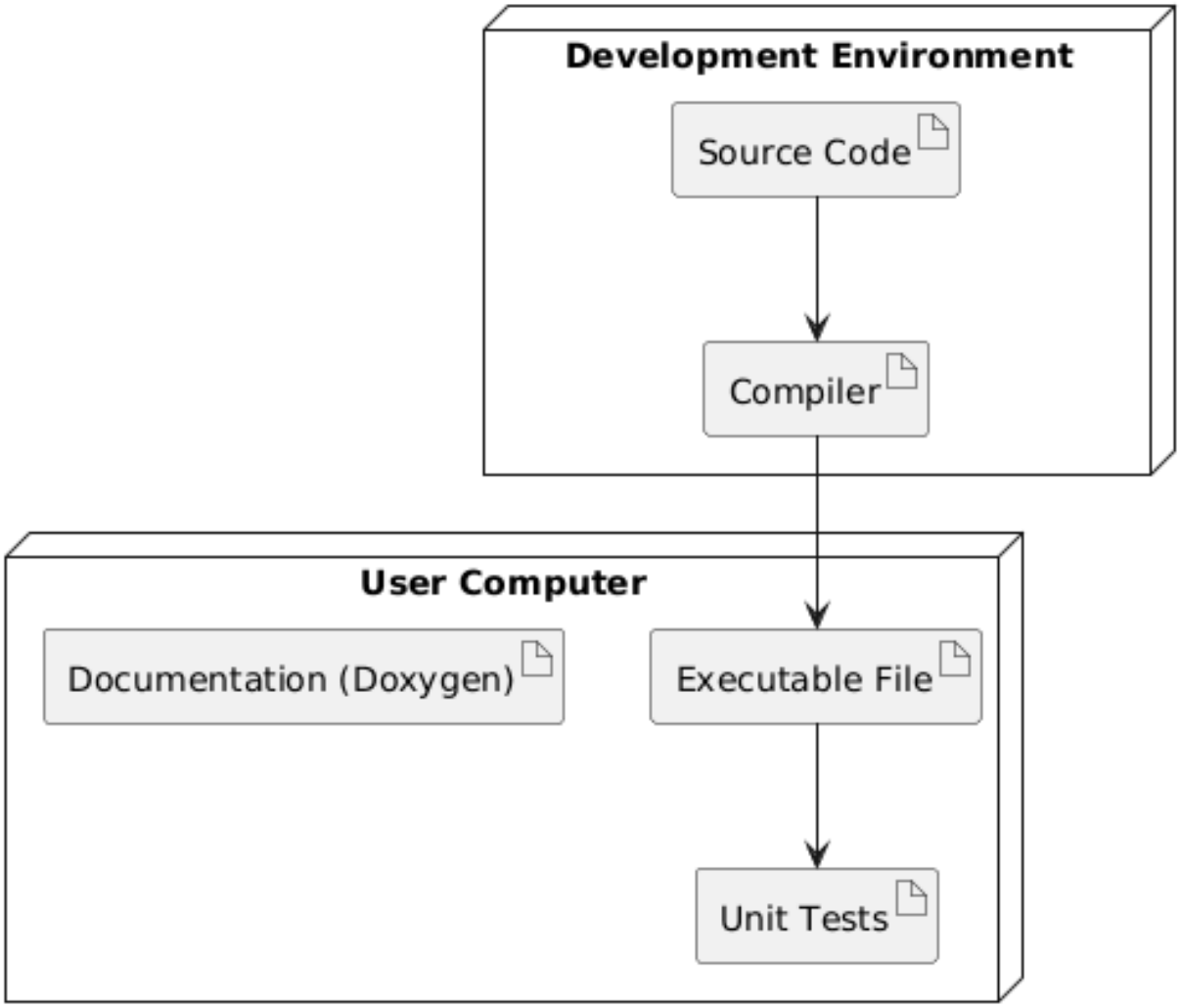


Deployment Diagram

Описує розгортання системи:

- Локальна машина
- Компілятор
- Виконуваний файл
- Тестове середовище

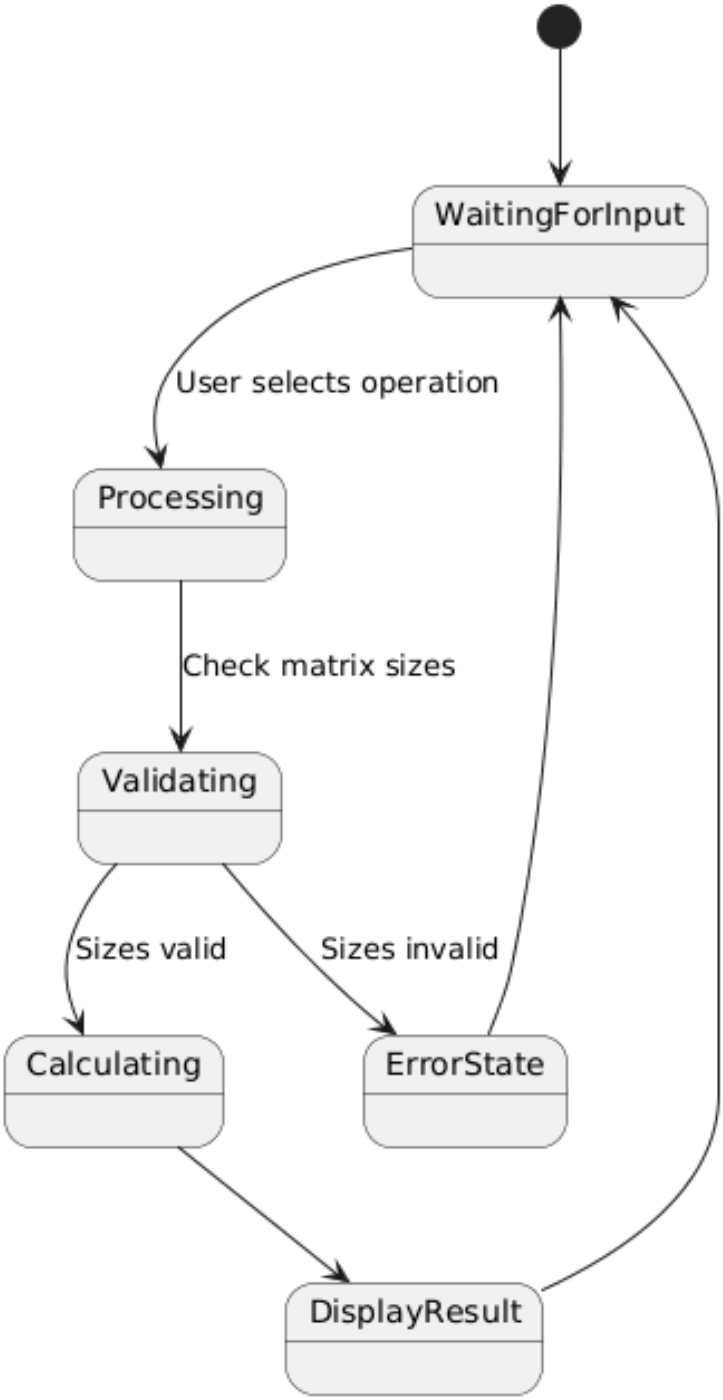
Система запускається локально.



State Diagram

Описує стани системи:

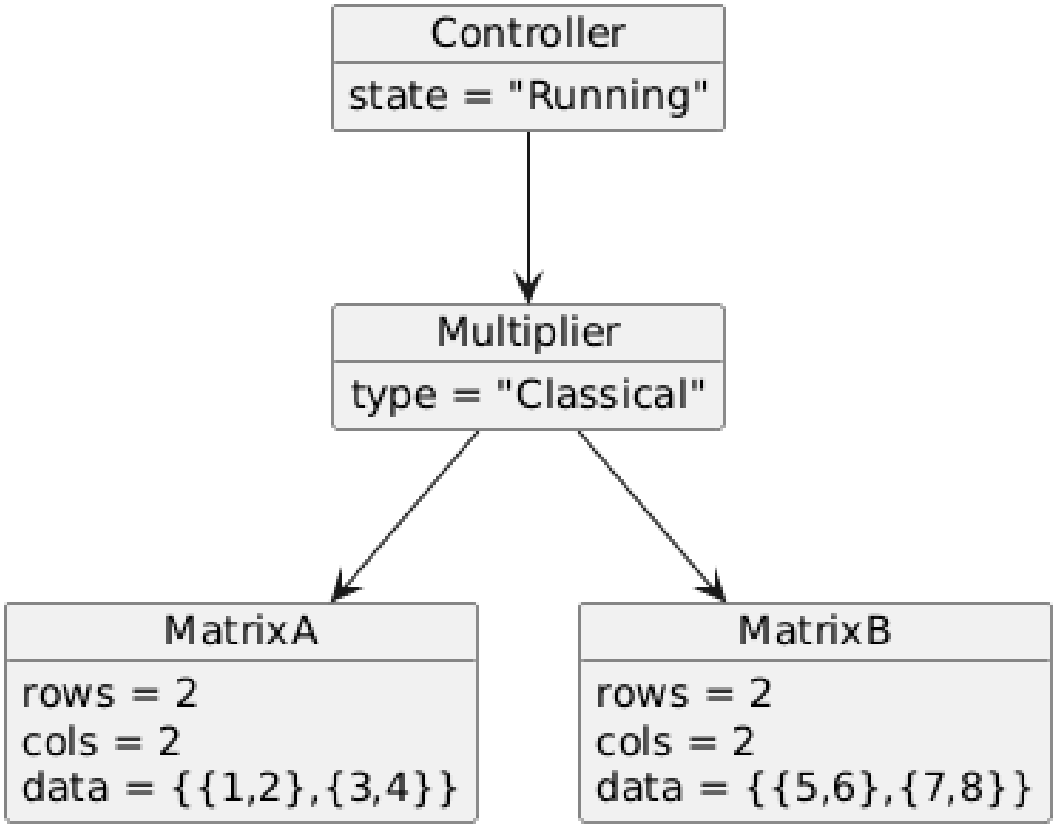
- Очікування введення
- Обробка операції
- Виконання алгоритму
- Виведення результату
- Повернення в меню



Object Diagram

Показує приклад об’єктів у пам’яті під час виконання:

- Об’єкт Matrix A
- Об’єкт Matrix B
- Об’єкт Multiplier
- Об’єкт Controller



Юніт-тестування

Для перевірки функціональності було реалізовано unit-тести з використанням бібліотеки Catch2.

Тестуються:

- Додавання матриць
- Віднімання
- Множення
- Алгоритм Штрассена
- Транспонування
- Граничні випадки
- Переповнення
- Порожні матриці

Було створено окремі файли тестів для:

- Старої версії
- Нової версії

Тести забезпечують перевірку коректності після рефакторингу.

Порівняння старої та нової версії

Стара версія:

- Процедурний стиль
- Глобальні змінні
- Фіксований розмір масиву
- Відсутність інкапсуляції
- Складність підтримки

Нова версія:

- Об’єктно-орієнтована архітектура
- Інкапсуляція даних
- Поліморфізм
- Патерн Strategy
- Розділення логіки та UI
- Підвищена тестованість

Аналіз архітектури

У системі застосовано принципи:

Інкапсуляція

Дані матриці приховані всередині класу Matrix.

Поліморфізм

Реалізований через інтерфейс IMatrixMultiplier.

SRP (Single Responsibility Principle)

Кожен клас має одну відповідальність.

OCP (Open Closed Principle)

Можна додати новий алгоритм множення без зміни існуючого коду.

DRY

Відсутнє дублювання алгоритмів.

KISS

Архітектура залишається простою та зрозумілою.

Висновки

У ході виконання лабораторної роботи було:

- Виконано аналіз старої версії програми
- Проведено рефакторинг
- Реалізовано ООП архітектуру
- Застосовано патерн Strategy
- Побудовано UML діаграми
- Реалізовано unit-тести
- Забезпечено перевірку коректності

Результатом є покращена, модульна та розширювана система для роботи з матрицями.

1. Чому було обрано саме цей інструмент, а не аналоги?

Для побудови UML-діаграм було обрано інструмент PlantUML.

Основною причиною вибору є:

- Можливість зберігати діаграми у вигляді текстових файлів
- Підтримка автоматичної генерації з текстового опису
- Легка версія контролю через систему версіонування
- Безкоштовність та відкритий доступ

Альтернативні інструменти (StarUML, Visual Paradigm, Lucidchart тощо) мають графічний інтерфейс, але не забезпечують такої зручності у зберіганні діаграм у вигляді коду.

2. Наскільки просто було встановити та почати використовувати?

Інструмент було легко встановити / використати (або без встановлення — якщо це web-версія).

Основні кроки:

- Завантаження / підключення плагіну
- Налаштування середовища
- Створення першої діаграми
- Генерація зображення

Процес був інтуїтивно зрозумілим, проблем із запуском не виникло.

3. Наскільки корисною була документація?

Документація інструменту є достатньо детальною.

Було легко знайти інформацію про:

- Синтаксис UML
- Приклади діаграм
- Параметри налаштування
- Можливості розширення

Однак у деяких випадках прикладів для складних діаграм було недостатньо, тому доводилося шукати додаткову інформацію в інтернеті або аналізувати приклади.

4. Наскільки зрозуміло було використовувати інструмент?

Інтерфейс / синтаксис були зрозумілими.

Основні функції, які використовувались:

- Створення нової діаграми
- Додавання елементів
- Зв’язування компонентів
- Генерація зображення
- Експорт у формат PNG / SVG

Після кількох спроб використання робота з інструментом стала дуже комфортною.

5. Чи підтримує інструмент всі 14 типів UML 2.5?

Інструмент підтримує більшість основних UML-діаграм, але не всі 14 типів повністю.

Підтримуються:

- Use Case
- Class
- Activity
- Sequence
- Component
- Deployment
- State
- Object
- Package

Частина специфічних або рідко використовуваних діаграм може підтримуватися частково або потребувати ручної реалізації.

6. Чи всі можливості діаграм підтримуються?

Не всі можливості UML специфікації реалізовані повністю.

Обмеження:

- Деякі складні варіанти відношень можуть не підтримуватись
- Частина розширених налаштувань відсутня
- Менша гнучкість у кастомізації зовнішнього вигляду

Проте для навчальних та проєктних задач функціональності достатньо.

7. Чи використовувалась генерація коду або зворотна генерація?

Було використано:

- Генерацію зображень з текстових UML-файлів
- Генерацію UML з коду

8. Наскільки зручно було використовувати інструмент?

Інструмент є зручним, оскільки:

- Не потребує складних маніпуляцій з графічним інтерфейсом
- Дозволяє швидко редагувати діаграму

Мінус — потрібно пам’ятати синтаксис.

9. Чи була поведінка інструменту зрозумілою?

Поведінка була передбачуваною.

Помилки у синтаксисі одразу відображались у вигляді повідомлень, що дозволяло швидко виправити проблему.

Іноді виникали неточності у вирівнюванні або позиціонуванні елементів.

10. Чи виникали проблеми?

Основні проблеми:

- Помилки у синтаксисі
- Неправильне відображення складних зв’язків
- Обмеження у стилізації

Проблеми були вирішені через:

- Перевірку документації
- Перезапуск генерації
- Спрощення структури діаграми

11. Позитивні аспекти інструменту

- Простота використання
- Текстовий формат
- Швидка генерація діаграм
- Підтримка більшості UML-типів

12. Негативні аспекти

- Обмежена візуальна кастомізація
- Не всі UML-деталі підтримуються повністю
- Потрібно знати синтаксис

13. Що б я змінив після отриманого досвіду?

Після виконання роботи я б:

- Раніше спланував структуру діаграм
- Спочатку створив глосарій
- Розбив великі діаграми на менші
- Використовував модульну структуру файлів